This document contains the complete wiring schematics, hardware warnings, and source code required to construct the initial single-player laser tag receiver. The system features infrared damage detection, RGB visual feedback, auditory alarms, and an I2C LCD for real-time health telemetry.

System Wiring Matrix

Component	Component Pin	ESP32 Pin	Function / Notes
TSOP1838 IR Receiver	Left Leg (OUT / Data)	**14** (D14)	Signal input
	Middle Leg (GND)	**GND**	System ground
	Right Leg (VCC)	**3V3** (3V)	Must use 3.3V logic
RGB LED Module	R (Red)	**25**	Damage indicator
	G (Green)	**26**	Healthy indicator
	B (Blue)	**27**	Unused in v1.0
	- (or GND)	**GND**	Assuming Common Cathode
16x2 I2C LCD Display	GND	**GND**	System ground
	VCC	**V5 / VIN / 5V**	Requires 5V for backlight
	SDA (Data)	**21**	Hardware I2C Data
	SCL (Clock)	**22**	Hardware I2C Clock
Active Buzzer	+ (Signal / IO)	**33**	Auditory alerts
	- (GND)	**GND**	System ground

Hardware Preparation Notes

> **LCD Blank Screen / Contrast Adjustment**

Software Dependencies

Before compiling, ensure the following libraries are installed via the Arduino IDE Library Manager:
1. **IRremoteESP8266** (by crankyoldgit)
2. **LiquidCrystal I2C** (by Frank de Brabander)

Firmware Source Code (Part 1/3)

LaserTag_Main.ino - Declarations & Setup

```
#include <Arduino.h>
#include <IRremoteESP8266.h>
#include <IRrecv.h>
#include <IRutils.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// --- PIN SETUP ---
const uint16_t IR_PIN = 14;
const int RED_PIN = 25;
const int GREEN_PIN = 26;
const int BLUE_PIN = 27;
const int BUZZER_PIN = 33;
// LCD uses default I2C pins: SDA = 21, SCL = 22

// --- LCD SETUP ---
LiquidCrystal_I2C lcd(0x27, 16, 2);

// --- REMOTE HEX CODES ---
// Codes that do nothing (Shields/Misses)
#define BTN_1 0xFF30CF
#define BTN_2 0xFF18E7
#define BTN_3 0xFF7A85
#define BTN_4 0xFF10EF

// Codes that cause damage (Hits)
#define BTN_5 0xFF38C7
#define BTN_6 0xFF5AA5
#define BTN_7 0xFF42BD
#define BTN_8 0xFF4AB5
#define BTN_9 0xFF52AD

IRrecv irrecv(IR_PIN);
decode_results results;

int hitCount = 0;
bool isEliminated = false;

// Helper to change LED colors
void setLED(bool r, bool g, bool b) {
  digitalWrite(RED_PIN, r);
  digitalWrite(GREEN_PIN, g);
```

Firmware Source Code (Part 2/3)

LaserTag_Main.ino - Initialization & Display Helpers

```
// Helper to update the LCD screen state
void updateDisplay() {
  lcd.clear();
  if (hitCount == 0) {
    lcd.setCursor(0, 0);
    lcd.print("Status: HEALTHY");
    lcd.setCursor(0, 1);
    lcd.print("Lives: 5");
  } else if (hitCount >= 5) {
    lcd.setCursor(0, 0);
    lcd.print("--- PLAYER ---");
    lcd.setCursor(0, 1);
    lcd.print(" ELIMINATED! ");
  } else {
    lcd.setCursor(0, 0);
    lcd.print("Status: DAMAGED");
    lcd.setCursor(0, 1);
    lcd.print("Lives: ");
    lcd.print(5 - hitCount);
  }
}

void setup() {
  Serial.begin(115200);
  irrecv.enableIRIn();

  pinMode(RED_PIN, OUTPUT);
  pinMode(GREEN_PIN, OUTPUT);
  pinMode(BLUE_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  // Initialize the LCD
  lcd.init();
  lcd.backlight();
  updateDisplay();

  // Default State: GREEN
  setLED(LOW, HIGH, LOW);
}

void loop() {
  if (isEliminated) {
    return; // Game over, freeze everything
  }

  if (irrecv.decode(&results)) {
    uint64_t code = results.value;

    // Ignore held-down button codes
    if (code == 0xFFFFFFFF || code == 0) {
      irrecv.resume();
```

Firmware Source Code (Part 3/3)

LaserTag_Main.ino - Main Loop (Damage & Cooldown)

```
    // Shield / Misses (Buttons 1-4)
    if (code == BTN_1 || code == BTN_2 || code == BTN_3 || code == BTN_4) {
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Attack Blocked!");
      delay(1000);
      updateDisplay(); // Restore normal health screen
    }

    // Damage (Buttons 5-9)
    else if (code == BTN_5 || code == BTN_6 || code == BTN_7 || code == BTN_8 || code
== BTN_9) {
      hitCount++;

      if (hitCount >= 5) {
        // Elimination Sequence
        updateDisplay();
        for (int i = 0; i < 3; i++) {
          setLED(HIGH, HIGH, LOW);
          digitalWrite(BUZZER_PIN, HIGH);
          delay(500);
          setLED(LOW, LOW, LOW);
          digitalWrite(BUZZER_PIN, LOW);
          delay(500);
        }
        isEliminated = true;
      } else {
        // Take Damage Sequence
        updateDisplay();
        setLED(HIGH, LOW, LOW);
        digitalWrite(BUZZER_PIN, HIGH);
        delay(200);
        digitalWrite(BUZZER_PIN, LOW);

        // Visual Cooldown Timer on LCD
        for(int i = 4; i > 0; i--) {
          lcd.setCursor(0,0);
          lcd.print("COOLDOWN: ");
          lcd.print(i);
          lcd.print("s   ");
          delay(1000);
        }
        delay(800); // Finish out the 4.8s delay

        setLED(LOW, HIGH, LOW); // Back to GREEN
        updateDisplay();
      }
    }
    irrecv.resume();
  }
}
```